Parallel and HPC
HW9 Report

Link to GITHUB repository:

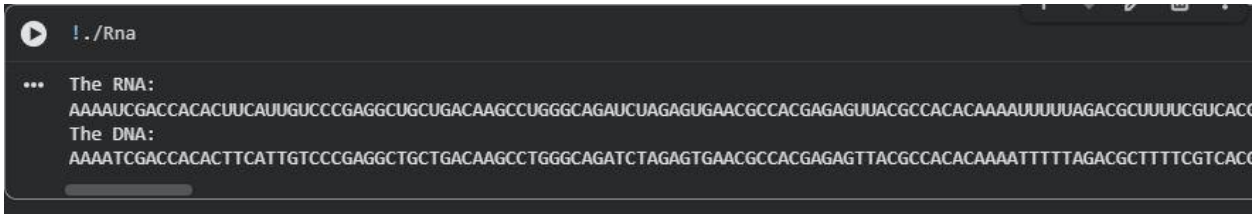
https://github.com/DMelkonyan111/Parallel-and-HPC-Homeworks

Exercise 1

The main function creates a DNA sequence that's one thousand letters long and fills it with random letters like A, C, G and T. This sequence is stored in a special kind of memory on the computer called pinned host memory. Then it is copied to a kind of memory on the computer. The computer makes a list with four spots to count how many A, C, G and T letters are in the sequence. The dna_count part of the program is like a worker that looks at each letter in the sequence. Each worker figures out which letter it is supposed to look at and makes sure it is supposed to be looking at a letter. If it is the worker looks at the letter. Adds one to the correct spot in the list. There are sixty four workers in each group and the computer figures out how groups it needs to look at all the letters. After all the workers are done the computer puts the list back in the memory and prints out how many A, C, G and T letters are, in the DNA sequence. The main function does this with the DNA sequence and the list of A, C, G and T counts. The DNA sequence is used to count the A C, G and T letters.

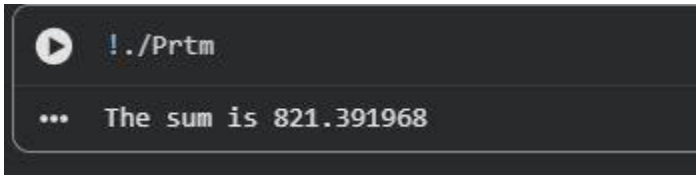
Exercise 2

The main function creates an RNA sequence that's 1000 characters long. It fills this sequence with characters from the set {A, C, G, U} and stores it in the host memory. The original RNA sequence is displayed. Then it is copied from the host memory to the device memory. In the rna_to_dna kernel each thread finds out its ID and checks if it is within the sequence bounds. If it is the thread checks if the current character is U. If the character is U it is replaced with T directly in the memory. The kernel is run with 32 threads in each block. The number of blocks is calculated to cover the RNA sequence. After all threads finish the modified sequence is copied back, to the host memory. The resulting DNA sequence is then displayed.



Exercise 3

The main part of the program has a string of protein "SKADYEK". It sends this string to a function called prtm. This function first finds out how long the string is. Then it copies the protein sequence from the computer memory to a special memory that the computer uses to do big jobs. It also makes a place to store the total mass of the protein. The program then uses something called a kernel, which's like a little worker to do the job. Each little worker finds out its special number, called a global thread ID and checks if it is supposed to be working on this job. If it is the worker looks at one of the amino acids in the protein string finds out how much it weighs and adds this weight to the weight. The program uses a lot of these workers 32 at a time and it figures out how many groups of workers it needs based on how long the protein string is. After all the workers are done the program waits for them to finish then it copies the weight back to the main computer memory and prints it out. The protein string "SKADYEK" is what this is, about so the program is really just trying to figure out the total mass of the protein "SKADYEK".



Exercise 5

The main function creates two sequences that're similar to RNA. These sequences are one thousand characters long. It fills them with letters like A, C, G and U. This is done using memory that is stored on the main computer. The sequences are then copied to memory spaces on a different device. A counter is also set up on this device to keep track of how differences there are between the sequences. Each part of the program called a thread figures out its identity and makes sure it is working within the correct boundaries. If it is the thread compares the characters at the spot in both sequences. When the characters are not the same the counter is increased. This is done in a way to make sure everything works correctly. The program is set up to use 32 threads at a time. It uses as many of these groups as it needs to check all the characters. After everything is done the final count of differences is sent back, to the computer and printed out. The main function and the kernel, which is called hamm work together to make this happen. The kernel hamm does the comparison of the RNA-like sequences.